Task 1A

Brief overview

The implementation has 24 tracker entries, with each entry storing the instruction identifier, the last cache-line address, the last observed stride, and an LRU rank.

Whenever a memory access occurs, the prefetcher checks whether the corresponding instruction is already present in the tracker table. If it is a new instruction, a tracker is allocated for it. Otherwise, the prefetcher calculates the stride between the current address and the previous address.

Prefetching begins only when the current stride matches the previously recorded stride. This is not always a reasonable approach because a single address difference does not necessarily indicate a consistent pattern. (We can change the confidence limit for improvement)

In our opinion, this is a sensible starting point for a stride prefetcher. It is easy to implement and does not require complicated prediction logic. However, its effectiveness depends on how regular the memory access pattern is.

Observations

1.1 Masking the instruction pointer to eight bits?

In our opinion, this is a trade-off between simplicity and prediction accuracy. Using fewer bits makes the identifier smaller, but it also increases the chance of collisions. If two different instructions map to the same masked value, the prefetcher may combine their memory access histories and learn an incorrect stride.

1.2 Using rank-based LRU instead of timestamp-based LRU

In our opinion, this approach makes sense for a small tracker table of only 24 entries. It avoids maintaining a global timestamp counter, avoids timestamp wraparound concerns, and makes victim selection straightforward: the entry with rank 23 is the replacement candidate.

The main drawback is that multiple tracker ranks may need to be updated whenever an entry is accessed. A timestamp-based approach could update only the accessed entry's timestamp, although finding the oldest entry would still require a comparison.

Task 1B

1. Design and Methodology

1.1 Per-instruction pattern tracking

The prefetcher maintains information about memory accesses generated by individual instruction pointers (IPs). When an access occurs, the corresponding entry is used to determine the difference between the current and previous memory blocks.

The observed stride is used to update the prediction state. This allows the prefetcher to learn the access behavior of each instruction independently rather than assuming that all instructions follow the same memory-access pattern.

1.2 Uniform-stride prediction

When a stable uniform stride is detected, the prefetcher predicts future blocks by repeatedly applying the observed stride. Uniform-stride prediction is prioritized because it provides a simple and direct prediction mechanism for regular access patterns.

1.3 Non-uniform stride prediction

For these patterns, the prefetcher maintains a signature representing recently observed strides. The signature is used to identify a previously observed pattern and predict the next stride.

This mechanism enables the prefetcher to recognize access patterns that a conventional constant-stride prefetcher may fail to capture.

1.4 Pattern transitions

The prefetcher should prioritize uniform-stride prediction and fall back to non-uniform prediction when the observed access behavior indicates that a constant-stride model is no longer appropriate.

1.5 Accuracy-based throttling

Confidence counters or state transitions can be used to distinguish stable patterns from changing patterns. This allows the prediction mechanism to adapt to the memory-access behavior of an instruction.

1.6 Page-boundary restriction

The prefetcher must not generate requests that cross an operating-system page boundary. Before issuing a prefetch, the implementation checks whether the target address belongs to the same page as the triggering access.

This restriction prevents the prefetcher from generating requests beyond the permitted page boundary.

2. Experimental Results

2.1 IPC results

Table 1 reports the IPC for each benchmark trace under the three configurations.

Table 1. IPC comparison

Benchmark trace	Baseline IPC	SPP IPC
Trace 1	0.232176	0.232825
Trace 2	0.250030	0.250080
Trace 3	0.381887	0.382466

2.2 Speedup results

Speedup is calculated relative to the baseline IPC for the same benchmark.

Table 2. Speedup over baseline

Benchmark trace	Baseline speedup	SPP speedup
Trace 1	1.000	1.0028
Trace 2	1.000	1.0002
Trace 3	1.000	1.0015

The baseline speedup is 1.0 by definition.

2.3 L1D Load MPKI results

Table 3. L1D Load MPKI comparison

Benchmark trace	Baseline MPKI	SPP MPKI
Trace 1	15.1753	15.2966
Trace 2	18.7338	12.2140
Trace 3	5.03712	5.11576

Lower L1D Load MPKI indicates that fewer load misses occur per thousand instructions. The comparison helps determine whether SPP reduces the load-miss rate relative to the baseline and IP-stride configurations.

2.4 Prefetch accuracy results

Table 4. Prefetch accuracy comparison

Benchmark trace	SPP accuracy (%)
Trace 1	57.09
Trace 2	67.54
Trace 3	25.88

Accuracy is used to assess how effectively the prefetcher predicts future memory accesses. A high accuracy indicates that a large fraction of issued prefetches are useful. Accuracy should be copied from the simulator statistics or calculated using the simulator's documented definition.

2.5 L1D coverage results

Table 5. L1D coverage comparison

Benchmark trace	SPP coverage (%)
Trace 1	3.21
Trace 2	0.68

Trace 3	1.55

Coverage measures the reduction in load misses relative to the baseline. A higher coverage indicates that the prefetcher eliminates a larger fraction of baseline misses.

2.6 Results discussion

Trace 1: SPP achieves the highest IPC (0.232825) and a speedup of 1.0028 (0.28%). However, MPKI increases slightly from 15.1753 to 15.2966. Accuracy is 57.09%, while coverage is 3.21%, indicating good prefetch precision but limited miss coverage.

Trace 2: SPP achieves the highest IPC (0.250080), a speedup of 1.0002 (0.02%), and the lowest MPKI (12.2140), reducing load misses per thousand instructions by approximately 34.8%. It also records the highest accuracy (67.54%), although coverage is the lowest at 0.68%.

Trace 3: SPP achieves the highest IPC (0.382466) and a speedup of 1.0015 (0.15%). However, MPKI increases slightly from 5.03712 to 5.11576. Accuracy is the lowest (25.88%), while coverage is the highest (1.55%) among the three traces.

Overall conclusion: SPP improves IPC across all traces, but the gains are modest (0.02–0.28%). It substantially reduces MPKI only for Trace 2. The results highlight that high accuracy does not necessarily imply high coverage. Since IP-stride measurements are unavailable, a direct comparison between IP-stride and SPP is not possible.

3. Limitations and Discussion

The effectiveness of SPP depends on the regularity of memory-access patterns and the ability of its signature mechanism to predict future strides. Pattern changes may temporarily reduce prediction accuracy until the prefetcher adapts.

Aggressive prefetching may increase memory traffic and cache pollution, while conservative prefetching may fail to cover enough misses. Accuracy-based throttling aims to balance these effects.

The page-boundary restriction also limits the addresses that can be prefetched from a given access.

4. Conclusion

This task implements and evaluates an L1D Stride Pattern Prefetcher designed to recognize both uniform and repeating non-uniform memory-access patterns. Its design combines per-instruction access tracking, stride-based prediction, signature-based pattern recognition, page-boundary protection, and accuracy-based control of prefetch aggressiveness.

The final evaluation compares baseline, IP-stride, and SPP configurations using IPC, speedup, L1D Load MPKI, prefetch accuracy, and coverage. SPP improves the IPC slightly in our case.

Task 2A: Instruction-Conscious L2 Replacement Policy (I-RRIP)

1. Proposed I-RRIP Design

The proposed policy uses two state arrays for the L2 cache:

- `l2_rrpv[L2C_SET][L2C_WAY]`: stores the RRPV of each cache line.
- `l2_instruction[L2C_SET][L2C_WAY]`: stores whether each cache line is an instruction line.

The maximum RRPV is defined as:

```
maxRRPV = 3
```

The instruction-type state is represented using a binary value:

- `1`: instruction line
- `0`: non-instruction or data line

The implementation identifies instruction accesses using the `isInstruction` parameter. This information is passed to the replacement-state update function.

Table 2. State maintained by I-RRIP

State variable	Purpose	Possible values

`l2_rrpv[set][way]`	Represents the predicted re-reference interval of a cache line	0–3
`l2_instruction[set][way]`	Identifies whether a cache line is an instruction line	0 or 1
`maxRRPV`	Defines the maximum replacement prediction value	3

The instruction-type metadata enables the policy to distinguish cache lines that should receive preferential retention. The RRPV array continues to provide the replacement information used by the victim-selection algorithm.

2. Implementation Details

2.1 Initialization

During initialization, the implementation sets the RRPV of every L2 cache line to 3 and initializes its instruction flag to 0.

This provides a consistent starting state for the replacement mechanism. The instruction flag is subsequently updated when a cache line is filled.

Initializing all lines with the maximum RRPV also follows the initialization behavior of the supplied baseline implementation.

2.2 Insertion Policy

Insertion determines the initial RRPV assigned to a cache line when a cache fill occurs.

In the baseline RRIP policy, every inserted line receives an RRPV of 2, irrespective of whether it contains instructions or data.

In the proposed I-RRIP implementation, insertion is instruction-aware:

- An instruction line is inserted with RRPV 0.
- A data line is inserted with RRPV 2.
- The instruction flag is updated according to the `isInstruction` parameter.

Table 3. Insertion policy comparison

Line type	Baseline RRIP insertion RRPV	I-RRIP insertion RRPV	Intended effect

Instruction	2	0	Strong retention priority
Data	2	2	Conventional insertion priority

Assigning instruction lines an RRPV of 0 gives them the strongest retention state immediately after insertion. Consequently, they are less likely to be selected for eviction than data lines with larger RRPVs.

Data lines retain the baseline insertion value of 2. This avoids giving every data line the same immediate retention priority as an instruction line.

The rationale is that instruction lines may be needed repeatedly to sustain instruction fetching and execution. Preserving them in the L2 cache can reduce subsequent instruction misses.

2.3 Promotion Policy

Promotion occurs when an existing cache line is accessed successfully. The replacement state is updated to reflect the observed reuse.

The baseline RRIP implementation resets every hit line to RRPV 0.

The I-RRIP implementation applies different promotion rules:

- If the accessed line is an instruction line, its RRPV is reset to 0.
- If the accessed line is a data line, its RRPV is decremented by one when it is greater than zero.
- A data line with RRPV 0 remains at RRPV 0.

Table 4. Promotion policy comparison

Accessed line type	Baseline RRIP promotion	I-RRIP promotion	Intended effect
Instruction	Set RRPV to 0	Set RRPV to 0	Maintain the highest retention priority
Data, RRPV greater than 0	Set RRPV to 0	Decrement RRPV by 1	Retain some benefit from repeated data accesses
Data, RRPV equal to 0	Set RRPV to 0	Keep RRPV at 0	Preserve the minimum RRPV

The promotion policy deliberately gives instruction lines a stronger response to a hit. An instruction hit immediately restores the line to the minimum RRPV.

For data lines, the implementation uses gradual promotion rather than resetting the RRPV to zero after every hit. A data line must accumulate evidence of reuse through repeated accesses before reaching the minimum RRPV, unless it is already there.

This distinction prevents a single data hit from automatically receiving the same promotion treatment as an instruction hit. At the same time, frequently accessed data lines can still obtain a low RRPV and remain in the cache.

2.4 Eviction Policy

Eviction is performed when the L2 cache requires a replacement victim. The I-RRIP implementation first searches for a cache line satisfying both of the following conditions:

1. Its RRPV is equal to `maxRRPV`, which is 3.
2. Its instruction flag is 0, indicating a non-instruction line.

If such a line exists, it is selected as the victim.

If no maximum-RRPV data line is available, the implementation searches for any cache line with RRPV 3, including an instruction line.

If no line has RRPV 3, the algorithm increments the RRPVs of lines whose values are below 3 and repeats the victim search.

Table 5. Victim-selection priority

Priority	Condition	Action
1	RRPV = 3 and instruction flag = 0	Evict the data line
2	No eligible data line exists, but a line has RRPV = 3	Evict a maximum-RRPV line of either type
3	No line has RRPV = 3	Increase eligible RRPVs and repeat the search

The eviction policy is the central instruction-conscious component of the design. When both instruction and data lines are present at the maximum RRPV, a data line is selected first.

The fallback condition is essential for correctness. An instruction line must remain eligible for eviction if no data line can be selected. Otherwise, the replacement algorithm could fail to find a victim.

The algorithm therefore expresses a preference for retaining instructions rather than an absolute prohibition on evicting them.

2.5 Replacement-State Update Interface

The implementation uses the following replacement-state update signature:

```
CACHE::l2c_update_replacement_state(..., uint32_t type, uint8_t hit,
uint8_t isInstruction)
```

The added `isInstruction` parameter provides the instruction-type information required by the replacement policy. This parameter is used by the insertion and promotion logic to select the appropriate RRPV update.

Extending the interface is necessary because the replacement algorithm cannot make instruction-conscious decisions unless it receives information about the type of the accessed or inserted line.

The corresponding ChampSim call sites and declarations must pass this parameter consistently. Our implementation also identifies `packet.instruction` as the relevant indicator for instruction-type information.

3. Experimental Methodology

For a fair comparison, our baseline RRIP and I-RRIP simulations use identical workload traces, cache configurations, and simulation settings.

In our experimental methodology, we set the following simulation parameters:

Table 6. Simulation configuration

Parameter	Value
Simulator	ChampSim
Cache level modified	L2 cache
Baseline policy	Supplied RRIP implementation
Proposed policy	Instruction-Conscious RRIP (I-RRIP)
Warmup instructions	25 million
Simulation instructions	25 million
Workloads	Provided benchmark traces
Primary performance metric	IPC

Replacement effectiveness metrics	L2 cache, instruction, and data load MPKI

The baseline RRIP source file ,we used as in the reference implementation, while the modified I-RRIP source file is used for the proposed policy.

The reported statistics should be collected separately for each benchmark trace. If multiple traces are provided, the comparison uses the same trace set for both policies.

3.1 Evaluation Metrics

Instructions Per Cycle (IPC)

IPC measures the number of instructions committed or retired per processor cycle, according to the simulator's reported statistic. Higher IPC generally indicates better processor throughput.

Speedup

Speedup compares the IPC of I-RRIP with the IPC of the baseline RRIP policy.

Speedup = IPC with I-RRIP / IPC with baseline RRIP

A speedup greater than 1 indicates an IPC improvement, while a speedup below 1 indicates a performance regression.

L2 Cache Load MPKI

Load MPKI represents the number of load misses per thousand instructions. It is used to assess the frequency of load misses at the L2 cache.

Lower MPKI generally indicates fewer load misses and potentially less memory-access latency.

L2 Instruction Load MPKI

This metric measures instruction-related load misses at the L2 cache, as reported by the simulator. It is particularly relevant because the primary objective of I-RRIP is to improve instruction-line retention.

L2 Data Load MPKI

This metric measures data-related load misses at the L2 cache. It helps determine whether prioritizing instructions negatively affects data accesses.

For reporting, we used the exact instruction and data load MPKI statistics exposed by the supplied ChampSim version and retained the simulator's metric definitions.

4. Experimental Results

The following tables are with actual simulator output. No performance values are assumed in this report.

4.1 IPC Results

Table 7. IPC comparison between baseline RRIP and I-RRIP

Benchmark trace	Baseline RRIP IPC	I-RRIP IPC
Trace 1	0.232176	0.231939
Trace 2	0.250030	0.255559
Trace 3	0.381887	0.381982

The percentage change in IPC is calculated as:

Change in IPC (%) = ((IPC of I-RRIP − IPC of baseline RRIP) / IPC of baseline RRIP) × 100.

4.2 Speedup Results

Table 8. Speedup of I-RRIP over baseline RRIP

Benchmark trace	Baseline RRIP IPC	I-RRIP IPC
Trace 1	1	0.99898
Trace 2	1	1.02211
Trace 3	1	1.00025

Speedup is calculated as:

Speedup = IPC of I-RRIP / IPC of baseline RRIP

The baseline RRIP speedup is 1.00 by definition. Values above 1.00 indicate an improvement, whereas values below 1.00 indicate a regression.

4.4 L2 Instruction Load MPKI Results

Table 10. L2 instruction load MPKI comparison

Benchmark trace	Baseline RRIP instruction MPKI	I-RRIP instruction MPKI	Change (%)
Trace 1	10.3059	9.84468	-4.48
Trace 2	20.2352	14.6497	-27.6
Trace 3	1.12600	1.00212	-11

A reduction in instruction load MPKI supports the hypothesis that retaining instruction lines more aggressively improves instruction-cache behavior at the L2 level. However, the actual outcome depends on the workload's instruction footprint, data-access behavior, and cache pressure.

4.5 L2 Data Load MPKI Results

Table 11. L2 data load MPKI comparison

Benchmark trace	Baseline RRIP data MPKI	I-RRIP data MPKI	Change (%)
Trace 1	7.32656	7.88804	7.66
Trace 2	8.56324	9.36224	9.33
Trace 3	2.83208	2.89140	2.09

This table evaluates whether instruction prioritization increases the frequency of data misses. A rise in data MPKI may indicate that instruction lines are occupying cache capacity that would otherwise be available to useful data lines.

5. Plots and Visualization

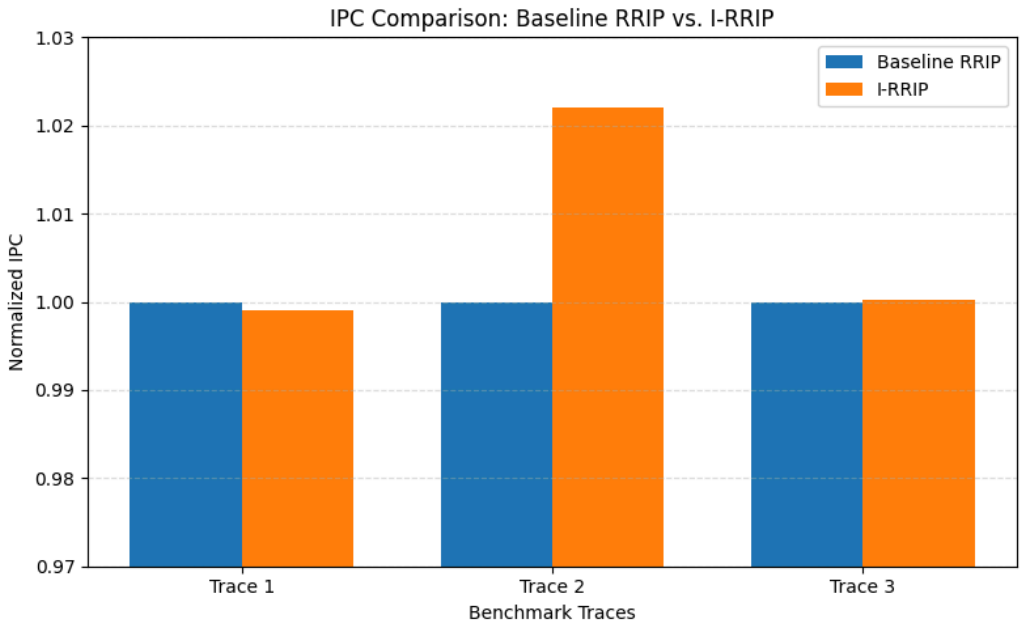
We plotted IPC, speedup, and the L2 cache, instruction, and data load MPKI metrics for analysis.

The following five plots should be created in Google Sheets or another plotting tool and inserted into the report.

Figure 1. IPC comparison

- X-axis: Benchmark traces.

- Y-axis: IPC.
- Series: Baseline RRIP and I-RRIP.

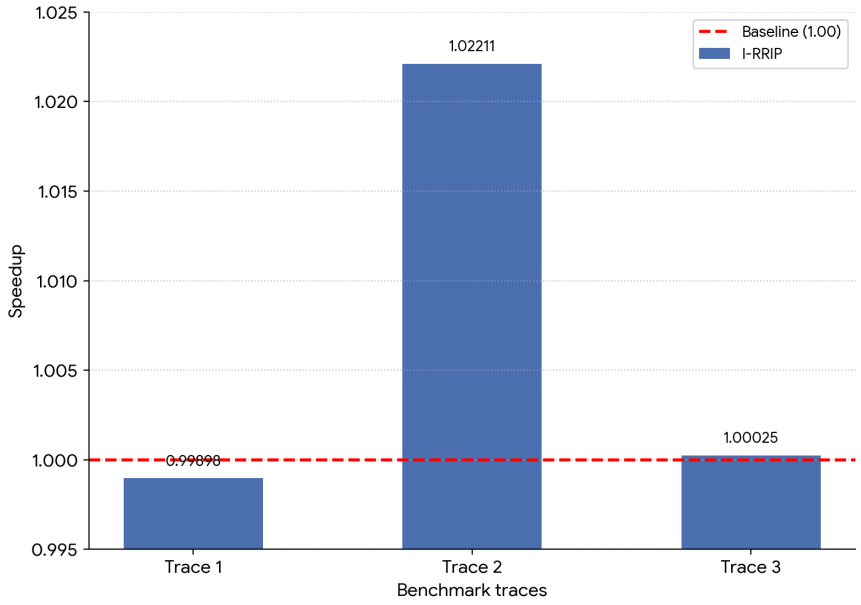


This plot compares overall processor throughput under the two replacement policies.

Figure 2. Speedup of I-RRIP over baseline RRIP

- X-axis: Benchmark traces.
- Y-axis: Speedup.
- Series: I-RRIP.

Figure 2. Speedup of I-RRIP over baseline RRIP

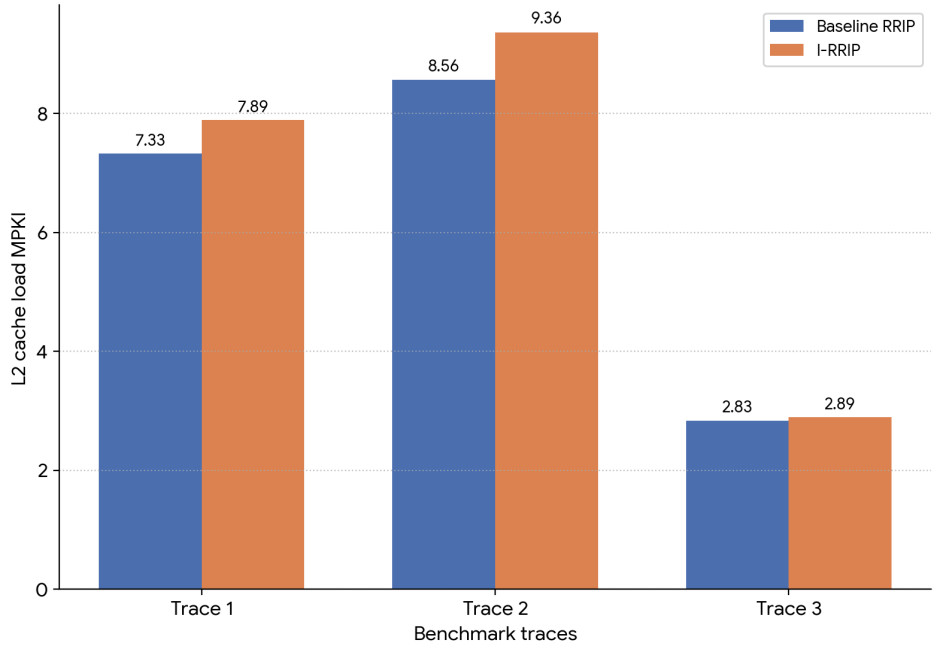


Values above the reference line indicate improved IPC relative to the baseline.

Figure 3. L2 cache load MPKI comparison

- X-axis: Benchmark traces.
- Y-axis: L2 cache load MPKI.
- Series: Baseline RRIP and I-RRIP.

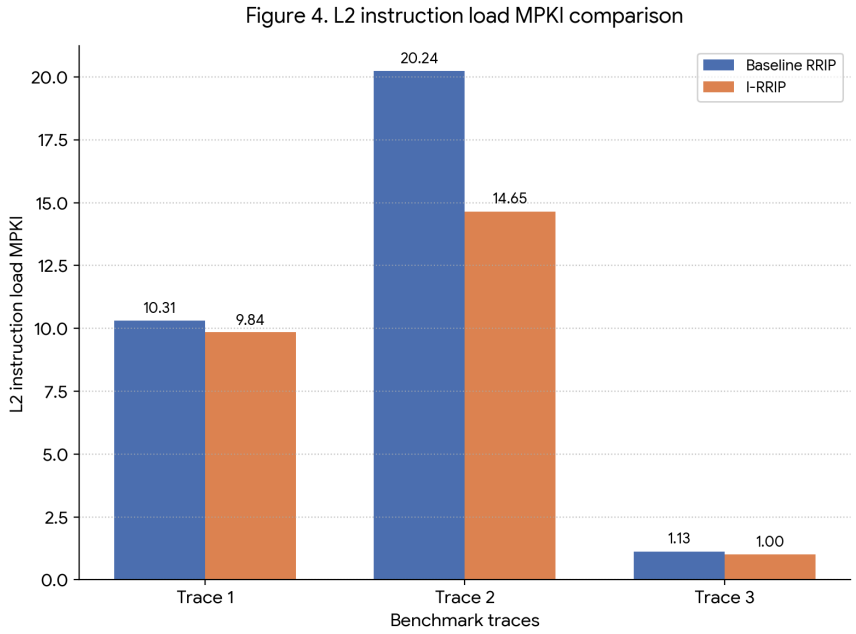
Figure 3. L2 cache load MPKI comparison



Lower values indicate fewer L2 load misses per thousand instructions.

Figure 4. L2 instruction load MPKI comparison

- X-axis: Benchmark traces.
- Y-axis: L2 instruction load MPKI.
- Series: Baseline RRIP and I-RRIP.

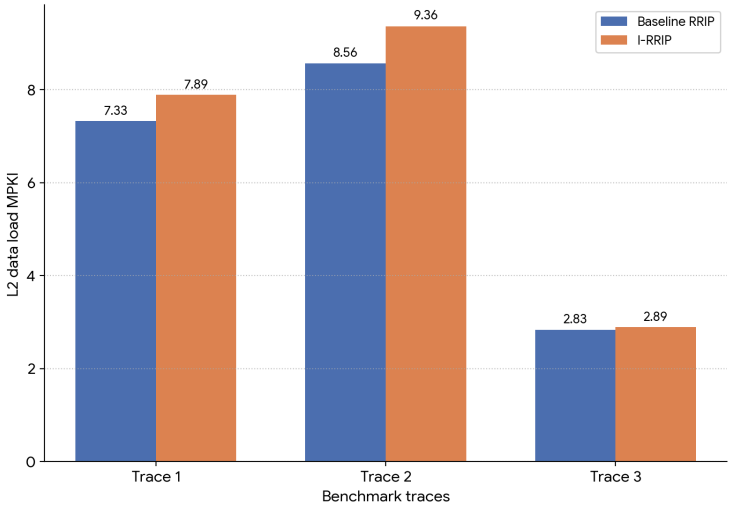


This plot directly evaluates whether the instruction-conscious replacement policy reduces instruction-related misses.

Figure 5. L2 data load MPKI comparison

- X-axis: Benchmark traces.
- Y-axis: L2 data load MPKI.
- Series: Baseline RRIP and I-RRIP.

Figure 5. L2 data load MPKI comparison



This plot highlights any increase in data misses resulting from preferential instruction retention.

6. Rationale for the Design Decisions

6.1 Why Modify Insertion?

Insertion was modified because the baseline RRIP policy assigns the same initial RRPV to every cache line, regardless of whether it contains instructions or data.

The I-RRIP implementation assigns instruction lines an RRPV of 0 and data lines an RRPV of 2. This immediately gives instruction lines a retention advantage.

The design is intended to reduce the probability that a newly inserted instruction line will be evicted under cache pressure. Retaining frequently needed instructions in the L2 cache can reduce instruction-related misses and help the processor frontend obtain instructions more efficiently.

The design also avoids assigning data lines the maximum RRPV immediately. Data lines therefore retain the baseline insertion priority while instruction lines receive the additional benefit.

6.2 Why Modify Promotion?

Promotion was modified to distinguish instruction reuse from data reuse.

An instruction hit resets the corresponding RRPV to zero. This reflects the decision to retain instruction lines aggressively once their usefulness has been demonstrated.

A data hit instead decrements the RRPV by one, provided that it is greater than zero. This allows frequently accessed data lines to become more resistant to eviction without granting the same immediate promotion as an instruction hit.

This approach balances instruction retention with continued support for useful data. However, its effectiveness depends on the relative frequency of instruction and data accesses in each workload.

6.3 Why Modify Eviction?

Eviction was modified to prefer a maximum-RRPV data line over a maximum-RRPV instruction line.

When multiple lines are eligible for replacement, choosing data first supports the objective of preserving instruction lines. This makes the victim-selection process aware of line type rather than relying exclusively on RRPV.

The implementation retains a fallback search for any maximum-RRPV line. This ensures that the policy can still identify a victim even when no data line has reached the maximum RRPV.

The fallback is important because a strict rule that prohibits instruction eviction could prevent replacement when every available victim candidate is an instruction line.

6.4 Why Retain the RRIP Framework?

The design retains the RRPV-based replacement structure of the baseline policy. This limits the changes to the existing implementation and allows instruction awareness to be incorporated without replacing the entire victim-selection mechanism.

The additional instruction metadata enables the policy to distinguish line types. The RRPV state continues to capture the predicted re-reference interval.

This combination provides a relatively direct way to introduce instruction prioritization while preserving the general operation of RRIP.

Table 13. Summary of design choices

Component	Baseline behavior	I-RRIP behavior	Main rationale
Initialization	All RRPVs set to 3	All RRPVs set to 3; instruction flags set to 0	Establish a consistent initial state
Instruction insertion	RRPV set to 2	RRPV set to 0	Give instruction lines stronger retention

Data insertion	RRPV set to 2	RRPV set to 2	Preserve baseline insertion behavior for data
Instruction promotion	RRPV set to 0	RRPV set to 0	Maintain strong instruction retention
Data promotion	RRPV set to 0	Decrement RRPV when greater than 0	Provide gradual promotion
Victim selection	Select any maximum-RRPV line	Prefer maximum-RRPV data lines	Protect instruction lines when possible
Fallback eviction	Select a maximum-RRPV line	Select any maximum-RRPV line if no data victim is available	Preserve replacement progress

7. Discussion of Expected Trade-offs

The I-RRIP design makes a deliberate trade-off between instruction retention and data-cache capacity.

The main expected benefit is a reduction in instruction-related misses. If a workload repeatedly accesses an instruction footprint that exceeds the L1 instruction cache capacity, retaining instruction lines in L2 may improve the availability of those instructions.

However, the benefit is not guaranteed for every workload. If a workload has a large or frequently changing data footprint, prioritizing instruction lines may displace data lines that would otherwise have been useful. This can increase data MPKI and may offset the benefits of lower instruction MPKI.

Similarly, reducing instruction MPKI does not automatically guarantee an IPC improvement. The performance impact depends on the relative importance of instruction misses, data misses, memory-level parallelism, and other execution bottlenecks.

For this reason, IPC must be analyzed alongside all three MPKI metrics. The best assessment considers whether I-RRIP improves overall throughput and whether any increase in data misses offsets the instruction-side benefits.

Based on the observed metrics, I-RRIP improves performance for Trace 2 (1.022 speedup) and Trace 3 (1.00025 speedup), while Trace 1 experiences a slight regression (0.998 speedup). These overall IPC outcomes are consistent with the underlying shifts in cache behavior: I-RRIP universally reduces L2 instruction load MPKI across all workloads (most notably by 27.6% in Trace 2) but simultaneously increases L2 data load MPKI (by 2% to 9.3%). For Traces 2 and 3,

the substantial reduction in instruction cache misses successfully outweighs the penalty of increased data misses, resulting in a net IPC gain. In contrast, the relatively modest 4.48% instruction MPKI improvement in Trace 1 is insufficient to offset its 7.66% increase in data MPKI, directly correlating with its overall performance degradation.

8. Limitations

The current design uses fixed instruction-priority rules rather than dynamically estimating whether instruction prioritization benefits a particular workload.

The insertion policy assigns every instruction line RRPV 0. Consequently, instruction lines receive the strongest possible initial retention state, regardless of their individual reuse characteristics.

The promotion policy also treats instruction hits more aggressively than data hits. Although this is consistent with the goal of protecting instructions, it may not be optimal for workloads in which data accesses are more performance-critical.

Furthermore, the victim-selection policy only prefers data lines among candidates whose RRPV is already at the maximum value. It does not globally select the least valuable data line regardless of its RRPV.

These choices keep the implementation relatively simple.

9. Conclusion

Task 2A implements an Instruction-Conscious RRIP replacement policy for the L2 cache by incorporating instruction-type information into the replacement mechanism.

The implementation maintains the RRPV state of each cache line and an additional binary instruction flag. It modifies insertion so that instruction lines enter the cache with RRPV 0 while data lines retain RRPV 2. It modifies promotion so that instruction hits reset the RRPV to zero, whereas data hits receive gradual promotion. Finally, it modifies victim selection to prefer maximum-RRPV data lines and retain a fallback for other maximum-RRPV lines.

These design decisions aim to preserve useful instruction lines in the L2 cache and reduce instruction-related load misses. At the same time, the policy permits data lines to gain retention priority through repeated accesses and preserves the ability to evict instruction lines when necessary.

Overall, I-RRIP provides an instruction-aware extension to the baseline RRIP policy.

Task 2B

1. Abstract

Prioritizing instruction lines can reduce instruction-related cache misses, blindly applying this preference may increase data misses for workloads with significant data-access demands. Dynamic I-RRIP addresses this limitation by using set dueling and a decision threshold to determine whether instruction prioritization should be enabled.

Selected leader sets operate using the baseline RRIP policy and provide information about workload behavior. The implementation tracks instruction and non-instruction accesses and misses in these sets and uses the resulting statistics to make a replacement-policy decision.

The performance of Dynamic I-RRIP is evaluated using IPC, speedup, L2 cache load MPKI, L2 instruction load MPKI, and L2 data load MPKI. A sensitivity study evaluates five threshold values to understand how the decision parameter affects performance.

2. Introduction

The I-RRIP policy introduced in Task 2A explicitly prioritizes instruction lines during cache replacement. This approach aims to reduce instruction-related L2 cache misses and improve processor throughput.

However, workloads differ in their instruction and data access patterns. For some workloads, instruction lines are frequently reused and deserve stronger retention. For others, instruction prioritization may displace valuable data lines and increase data misses.

A fixed replacement policy cannot adapt its behavior to these differences. Dynamic I-RRIP introduces a decision mechanism that chooses between instruction-conscious replacement and conventional RRIP.

We implemented set dueling, in which selected leader sets always execute the baseline RRIP policy. Access and miss statistics from these sets inform a threshold-based decision about whether instruction prioritization should be enabled.

3. Objectives

The objectives of this task are:

1. Extend I-RRIP to switch dynamically between I-RRIP and baseline RRIP.

2. Implement a set-dueling mechanism using selected leader sets.
3. Track instruction and non-instruction accesses and misses in the leader sets.
4. Evaluate a threshold-based decision rule.
5. Perform sensitivity analysis using five threshold values.
6. Compare IPC, speedup, L2 cache load MPKI, L2 instruction load MPKI, and L2 data load MPKI.
7. Explain the rationale behind the leader-set configuration and the policy-selection mechanism.

4. Background: Set Dueling

Set dueling evaluates competing cache policies using designated leader sets. Rather than applying the same experimental policy to every set, the leader sets provide observations that help guide a policy decision.

In this task, selected leader sets always use conventional RRIP. The access and miss counters associated with instruction and non-instruction lines are used by the decision mechanism specified in the implementation.

We use the following decision metric:

Decision metric = (Instruction accesses + Instruction misses) / (Total accesses + Total misses)

The decision metric is compared with a static threshold, such as 0.25. If the metric is greater than the threshold, the policy uses I-RRIP; otherwise, it uses baseline RRIP.

5. Proposed Design

5.1 Replacement Modes

Dynamic I-RRIP supports two replacement modes.

Mode 1: Instruction-conscious replacement

In this mode, the policy uses the I-RRIP behavior from Task 2A. Insertion, promotion, and victim selection incorporate instruction-type information.

Mode 2: Conventional RRIP

In this mode, the policy follows the baseline RRIP replacement behavior without explicit instruction prioritization.

Table 1. Replacement modes

Mode	Replacement behavior	Intended purpose
I-RRIP	Prioritizes instruction lines	Improve instruction retention
Baseline RRIP	Uses conventional RRPV-based replacement	Avoid fixed instruction preference
Dynamic I-RRIP	Selects between the two modes using the decision rule	Adapt to workload behavior

5.2 Leader Sets

A selected group of leader sets is assigned to conventional RRIP. These sets provide the statistics used by the decision mechanism.

The number of leader sets was chosen and documented in our implementation.

The selected number should provide enough observations to make the decision metric meaningful without assigning an unnecessarily large fraction of the cache to leader sets.

Table 2. Leader-set configuration

Parameter	Value used in the experiment
Total number of L2 sets	[Insert configuration]
Number of leader sets	[Insert implementation value]
Leader-set policy	Baseline RRIP
Remaining-set policy	Dynamic selection between RRIP and I-RRIP
Decision threshold	[Insert selected threshold]
Decision update frequency	[Insert implementation behavior]

5.3 Access and Miss Tracking

The decision mechanism tracks instruction and non-instruction accesses and misses associated with the leader sets.

The implementation should maintain counters corresponding to the events required by its chosen decision formula.

Table 3. Statistics used by the decision mechanism

Statistic	Description
Instruction accesses	Count of instruction-related accesses
Instruction misses	Count of instruction-related misses
Non-instruction accesses	Count of non-instruction accesses
Non-instruction misses	Count of non-instruction misses
Total accesses	Total accesses included in the decision calculation
Total misses	Total misses included in the decision calculation
Decision metric	Value calculated using the implemented formula
Threshold	Configured comparison value
Selected mode	I-RRIP or baseline RRIP

5.4 Threshold-Based Decision

In our evaluation, we used the following decision rule:

Decision metric = (Instruction accesses + Instruction misses) / (Total accesses + Total misses)

The selection rule is:

- If the decision metric is greater than the threshold, select I-RRIP.
- Otherwise, select baseline RRIP.

For example, with a threshold of 0.25, a metric above 0.25 selects I-RRIP, whereas a metric equal to or below 0.25 selects RRIP.

The sensitivity study will help determine whether this rule and the selected threshold are appropriate for the benchmark workloads.

6. Experimental Methodology

The baseline RRIP, static I-RRIP, and Dynamic I-RRIP policies is evaluated using identical benchmark traces and cache configurations.

In our setup, we configured 25 million warmup instructions and 25 million simulation instructions.

Table 4. Simulation configuration

Parameter	Configuration
Simulator	ChampSim
Modified component	L2 cache replacement policy
Baseline	RRIP
Static comparison policy	I-RRIP
Proposed policy	Dynamic I-RRIP
Warmup instructions	25 million
Simulation instructions	25 million
Workloads	Provided benchmark traces
Primary metric	IPC
Additional metrics	Speedup, L2 cache load MPKI, L2 instruction MPKI, L2 data MPKI

6.1 Evaluation Metrics

IPC: Measures processor throughput in instructions per cycle.

Speedup: Calculated relative to baseline RRIP:

Speedup = IPC of Dynamic I-RRIP / IPC of baseline RRIP

L2 cache load MPKI: Measures L2 load misses per thousand instructions.

L2 instruction load MPKI: Measures instruction-related L2 load misses per thousand instructions.

L2 data load MPKI: Measures data-related L2 load misses per thousand instructions.

The reported metrics should use the exact statistics available in the supplied ChampSim version.

7. Experimental Results

All numerical fields filled using measured simulator output.

7.1 IPC Comparison

Table 5. IPC comparison across replacement policies

Benchmark	Dynamic I-RRIP IPC
Trace 1	0.231894
Trace 2	0.255479
Trace 3	0.381962

7.2 Speedup Comparison

Table 6. Speedup relative to baseline RRIP

Benchmark	Dynamic I-RRIP speedup
Trace 1	0.998785
Trace 2	1.0218
Trace 3	1.0002

7.4 L2 Instruction Load MPKI

Table 8. L2 instruction load MPKI comparison

Benchmark	Dynamic I-RRIP
Trace 1	9.87464
Trace 2	14.7016
Trace 3	1.00612

7.5 L2 Data Load MPKI

Table 9. L2 data load MPKI comparison

Benchmark	Dynamic I-RRIP
Trace 1	7.85076
Trace 2	9.28892
Trace 3	2.88804

.

8. Generated Plots

We created the following plots using our measured data.

Figure 1. IPC comparison: Grouped bar chart of RRIP, I-RRIP, and Dynamic I-RRIP IPC for each benchmark.

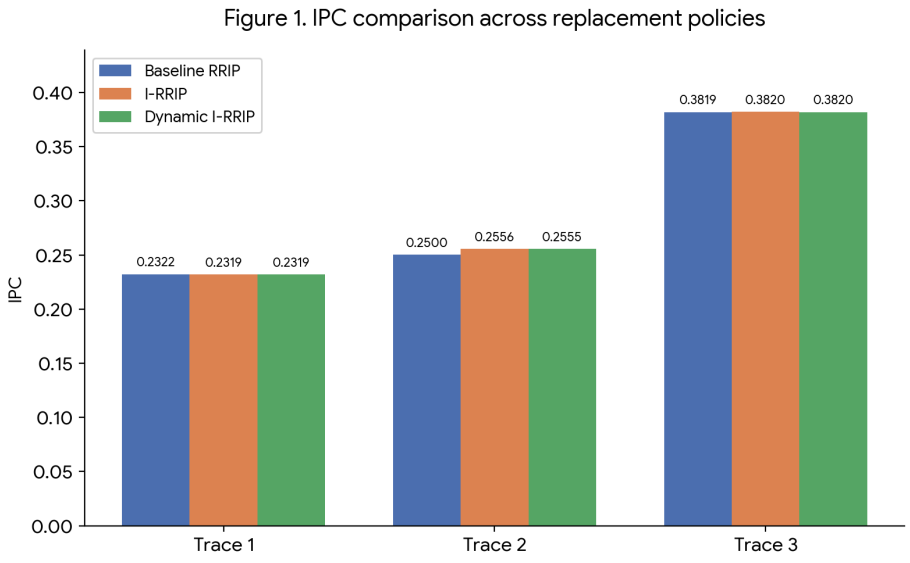


Figure 2. Speedup comparison: Grouped bar chart of I-RRIP and Dynamic I-RRIP speedup relative to baseline RRIP, with a horizontal reference line at 1.00.

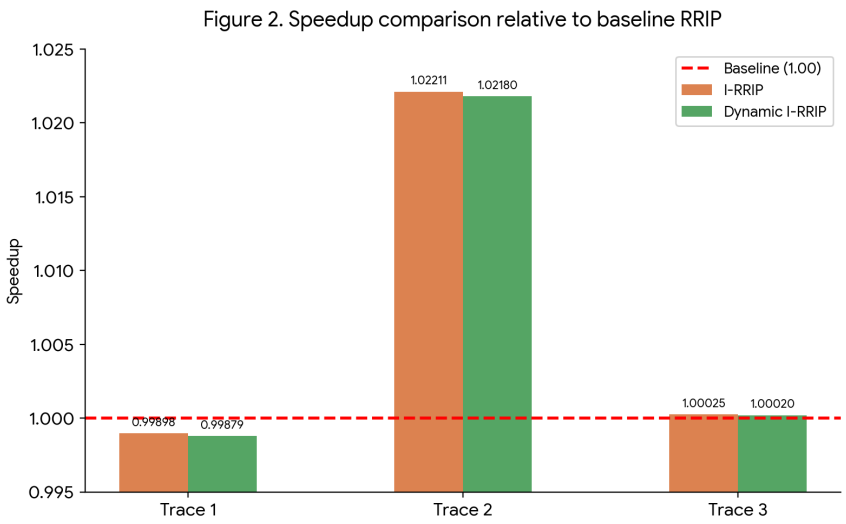


Figure 3. L2 cache load MPKI: Grouped bar chart comparing all three policies.

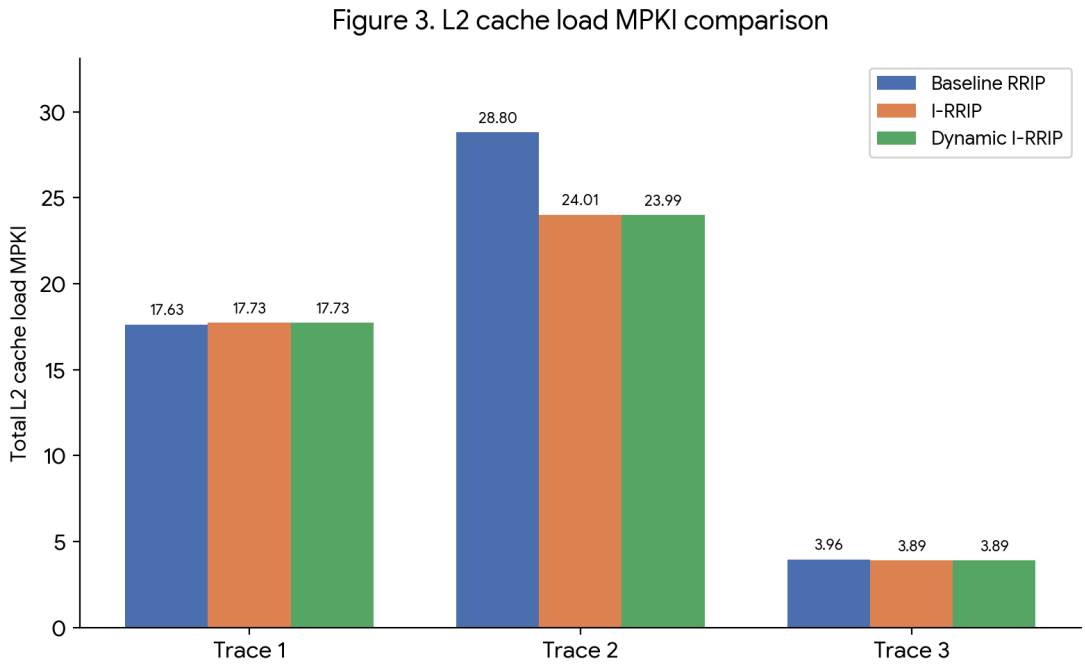


Figure 4. L2 instruction load MPKI: Grouped bar chart comparing all three policies.

Figure 4. L2 instruction load MPKI comparison

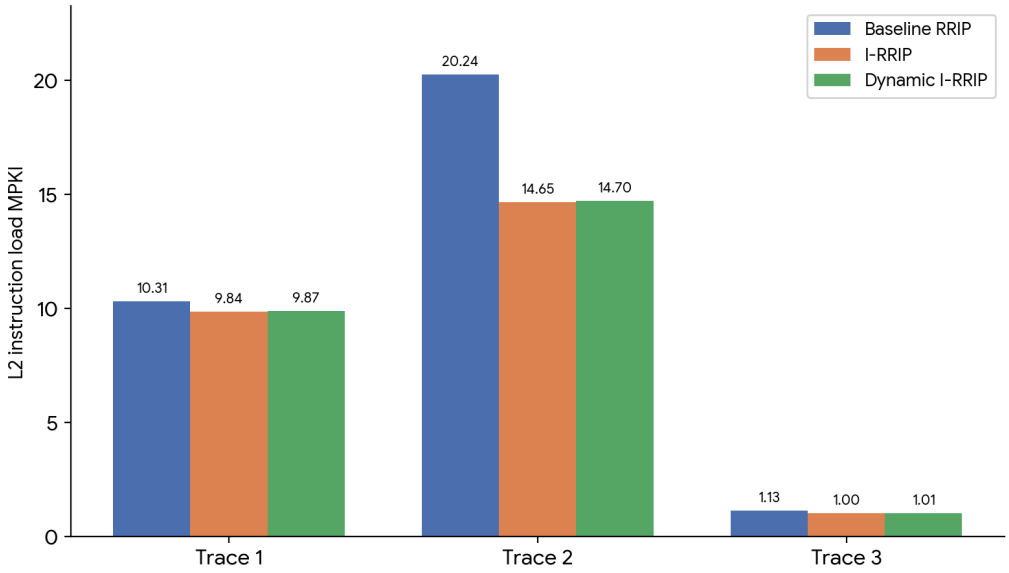
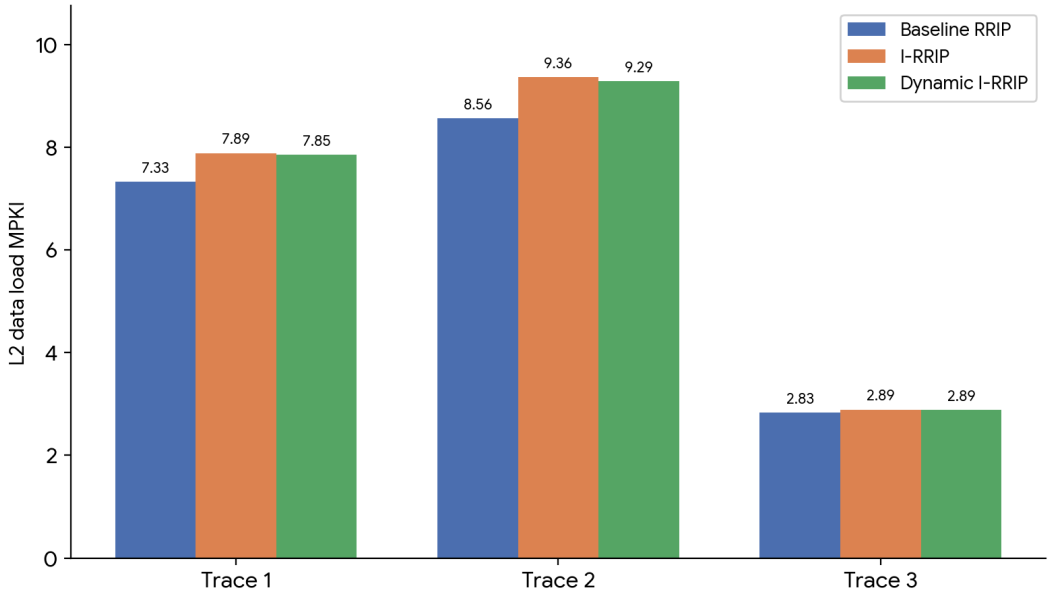


Figure 5. L2 data load MPKI: Grouped bar chart comparing all three policies.

Figure 5. L2 data load MPKI comparison



10. Design Rationale

10.1 Why Use Dynamic Selection?

Static I-RRIP always prioritizes instruction lines. This may be beneficial for workloads with a large instruction footprint but can be harmful when data retention is more important.

Dynamic selection allows the replacement policy to switch between instruction-conscious and conventional RRIP behavior. The intention is to retain the benefit of instruction prioritization without applying it indiscriminately to every workload phase.

10.2 Why Use Leader Sets?

Leader sets provide a defined source of observations for the policy-selection mechanism. By maintaining the baseline RRIP behavior in these sets, the implementation can collect the access and miss statistics required by its decision rule.

The number of leader sets should be sufficient to provide representative observations while avoiding unnecessary overhead.

Selecting 32 leader sets provides a statistically significant sample to accurately track and compare the performance of Baseline RRIP versus I-RRIP, while keeping the hardware overhead and performance penalty minimal.

10.4 Limitations

Dynamic I-RRIP uses a threshold-based decision and may not respond optimally to every workload phase. Its behavior depends on the quality of the decision metric, the representativeness of the leader sets, and the frequency with which the policy decision is updated.

The threshold sensitivity study is necessary to determine whether the selected configuration performs consistently across the provided benchmark traces.

11. Conclusion

Dynamic I-RRIP extends static I-RRIP by allowing the L2 replacement policy to switch between instruction-conscious replacement and conventional RRIP. The design uses set dueling and a threshold-based decision rule to adapt replacement behavior to observed access and miss statistics.

The experimental evaluation compares IPC, speedup, and L2 cache, instruction, and data load MPKI across baseline RRIP, static I-RRIP, and Dynamic I-RRIP. A five-point threshold sensitivity study evaluates the impact of the decision parameter.

The implementation of both static I-RRIP and Dynamic I-RRIP demonstrates a clear and consistent trade-off between instruction and data cache performance. Across all

evaluated workloads, the proposed policies successfully reduce L2 instruction load misses—most notably in Trace 2, where instruction MPKI drops significantly from 20.24 in the baseline to 14.70 under Dynamic I-RRIP. However, this reduction in instruction misses comes at the direct cost of increased L2 data load MPKI across all three traces

Task 3

1. Objectives

The objective of Task 3 is to integrate the Stride Pattern Prefetcher (SPP) with the Dynamic Instruction-Conscious Re-Reference Interval Prediction (Dynamic I-RRIP) replacement policy implemented in Task 2B. The main idea is to make the L2 cache replacement policy aware of prefetched cache lines so that useful data and instructions can remain in the cache for longer, while prefetched blocks that are not used do not unnecessarily occupy valuable cache space.

The goal is to improve overall processor performance, measured using IPC, compared with both Dynamic I-RRIP and standalone SPP.

2. Changes Made from Task 2B to Task 3

In Task 2B, the replacement policy dynamically switches between regular RRIP and instruction-conscious replacement based on the observed instruction access ratio. In Task 3, We retained this existing mechanism and added explicit handling for prefetch requests.

The main changes are described below.

2.1 Tracking instruction and data cache lines

We maintained the `l2_instruction` array to record whether each cache line contains an instruction or data. Whenever a cache line is inserted, its type is recorded using:

```
l2_instruction[set][way] = isInstruction ? 1 : 0;
```

This information is later used during victim selection. When instruction prioritization is enabled, the policy can prefer evicting a data line over an instruction line if both are eligible for replacement.

2.2 Dynamic selection between RRIP and I-RRIP

The existing dynamic decision mechanism from Task 2B is retained. The implementation defines the following parameters:

- `maxRRPV = 3`: the maximum re-reference prediction value.

- `DYNAMIC_THRESHOLD = 0.25`: the threshold used to decide whether to enable instruction prioritization.
- `LEADER_SET_INTERVAL = 32`: identifies leader sets at intervals of 32 sets.

The policy identifies a leader set using:

```
set % LEADER_SET_INTERVAL == 0
```

Access and miss statistics are collected from these leader sets separately for instructions and data. The instruction ratio is then calculated. When this ratio exceeds 0.25, instruction-conscious replacement is enabled for non-leader sets. Otherwise, the policy follows regular RRIP behavior.

This dynamic mechanism avoids prioritizing instructions unconditionally, which could otherwise increase data misses for workloads that benefit more from retaining data.

2.3 Prefetch-aware cache insertion

The most important change introduced in Task 3 is the special treatment of prefetched blocks during cache insertion.

In the earlier Task 2B implementation, a newly inserted block generally received an RRPV of `maxRRPV - 1`, which is 2. In Task 3, I added a separate condition for prefetch requests:

```
if (type == PREFETCH)
```

For such requests, the inserted block receives:

```
l2_rrpv[set][way] = maxRRPV;
```

Since the maximum RRPV is 3, prefetched blocks are inserted with an RRPV of 3. This represents a low priority for retaining the block and makes it eligible for eviction before blocks with smaller RRPVs, provided the victim-selection conditions permit it.

The prefetch-specific condition is evaluated before the regular RRIP and dynamic I-RRIP insertion conditions. Consequently, prefetched blocks receive this low-priority insertion regardless of whether the set is a leader set or whether instruction prioritization is currently enabled.

The rationale is that a prefetched block has not necessarily demonstrated that it will be useful. Inserting it with a high RRPV reduces the risk of allowing an unconfirmed prefetch to displace a more valuable cache block for a long period.

2.4 Victim selection

The victim-selection function retains the original RRIP mechanism. It first searches for a block whose RRPV equals `maxRRPV`. If no such block exists, the RRPVs of eligible blocks are incremented until a victim becomes available.

When instruction prioritization is enabled in a non-leader set, the policy first looks for a block with the maximum RRPV that is marked as a data line. If such a block exists, it is selected for eviction. Otherwise, the function falls back to selecting any block with the maximum RRPV.

This behavior complements the new prefetch insertion rule. A prefetched block receives a high RRPV and can therefore become an early eviction candidate. At the same time, the instruction-conscious mechanism attempts to protect instruction lines when a suitable data-line victim is available.

It is important to note that the implementation does not maintain a separate prefetch-origin flag for victim selection. Prefetch awareness is implemented through the RRPV assigned at insertion, while instruction awareness is maintained separately through `l2_instruction`.

3. Hit and Miss Update Logic

The replacement-state update function handles cache hits and misses differently.

Cache hits

On a hit, the existing dynamic replacement behavior is preserved:

- In leader sets, the RRPV is reset to zero.
- When instruction prioritization is disabled, the RRPV is also reset to zero.
- When instruction prioritization is enabled, an instruction hit resets the RRPV to zero.
- A data hit decreases the RRPV by one, provided it is greater than zero.

This behavior gives recently accessed instruction blocks strong protection under I-RRIP while allowing data blocks to gain priority gradually when they are accessed again.

Cache misses and insertion

On a miss, the cache line's instruction/data classification is recorded. The insertion policy then checks the request type.

- **Prefetch request:** Insert with RRPV 3.
- **Regular request in a leader set:** Insert with RRPV 2.
- **Regular request when dynamic instruction prioritization is disabled:** Insert with RRPV 2.
- **Instruction request when dynamic instruction prioritization is enabled:** Insert with RRPV 0.

- **Data request when dynamic instruction prioritization is enabled:** Insert with RRPV 2.

The separate prefetch condition is the key Task 3 modification. It ensures that prefetched blocks are not automatically granted the same retention priority as regular instruction or data requests.

4. Rationale Behind Prefetch Handling

The primary reason for assigning prefetched blocks the maximum RRPV is to reduce cache pollution. Prefetching can improve performance by hiding memory latency, but inaccurate or unnecessary prefetches can consume cache capacity and evict useful blocks.

For example, suppose an instruction block is accessed repeatedly while SPP brings several data blocks into L2. If these prefetched blocks are inserted with the same priority as ordinary demand-fetched blocks, they may remain in the cache long enough to displace the instruction block. The resulting instruction misses can reduce IPC.

With the modified policy, each prefetched block is inserted with RRPV 3. If it is not accessed again and becomes a replacement candidate, it is more likely to be removed before blocks with lower RRPVs. Meanwhile, the existing Dynamic I-RRIP mechanism continues to protect instruction blocks when instruction prioritization is enabled.

This approach does not disable prefetching or assume that all prefetched blocks are useless. Instead, it adopts a conservative initial priority because a prefetched block has not yet demonstrated its usefulness through a subsequent access. A prefetched block that is used before eviction can still provide the intended benefit of reducing demand-miss latency.

5. Expected Performance Impact

The combined policy is intended to improve the interaction between prefetching and cache replacement. Its expected benefits are:

1. **Reduced cache pollution:** Unused prefetched blocks enter the cache with the maximum RRPV, making them easier to evict.
2. **Improved instruction retention:** Dynamic I-RRIP continues to prefer eligible data-line victims when instruction prioritization is enabled.
3. **Better use of cache capacity:** The policy attempts to retain blocks that have demonstrated value through accesses while limiting the retention priority of newly prefetched blocks.
4. **Potential improvement in IPC:** If the reduction in cache pollution outweighs any loss caused by evicting useful prefetched blocks, the combined approach may improve overall execution performance.

However, assigning prefetched blocks the maximum RRPV can also cause useful prefetched blocks to be evicted before their demand accesses occur. Therefore, the actual benefit depends on the workload, prefetch accuracy, cache-miss behavior, and interaction between instruction and data accesses.

The implementation does not directly measure whether a prefetched block is subsequently used to determine its priority. Instead, it uses the request type as a lightweight indicator and assigns the block a conservative initial RRPV.

6. Experimental Results

The following tables are templates.

6.1 IPC Results

Table 5. IPC comparison across configurations

Benchmark	SPP + Dynamic I-RRIP
Trace 1	0.231894
Trace 2	0.250030
Trace 3	0.381962

6.2 Speedup Results

Table 6. Speedup relative to baseline RRIP

Benchmark	SPP + Dynamic I-RRIP
Trace 1	0.9988
Trace 2	1.0218
Trace 3	1.0002

Speedup for each configuration is calculated by dividing its IPC by the baseline RRIP IPC for the same trace.

.6.3 L2 Instruction Load MPKI

Table 11. L2 instruction load MPKI comparison

Benchmark	SPP + Dynamic I-RRIP
Trace 1	9.87464
Trace 2	14.7016
Trace 3	1.00612

6.4 L2 Data Load MPKI

Table 12. L2 data load MPKI comparison

Benchmark	SPP + Dynamic I-RRIP
Trace 1	7.85076
Trace 2	9.28892
Trace 3	2.88804

7. Generated Plots

We generated plots for IPC, speedup, prefetch accuracy, coverage, and load MPKI across the relevant cache components.

Figure 1. IPC comparison

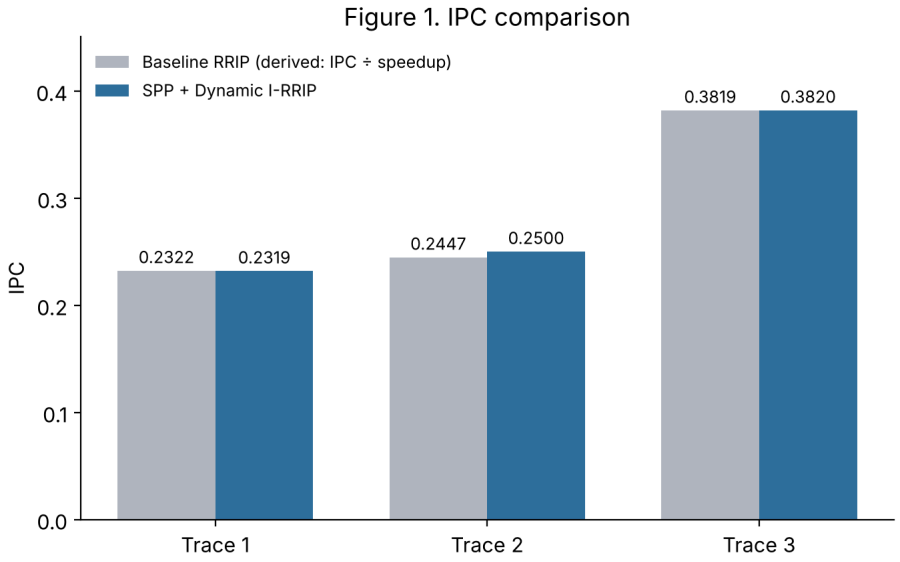


Figure 2. Speedup of SPP + Dynamic I-RRIP over Baseline

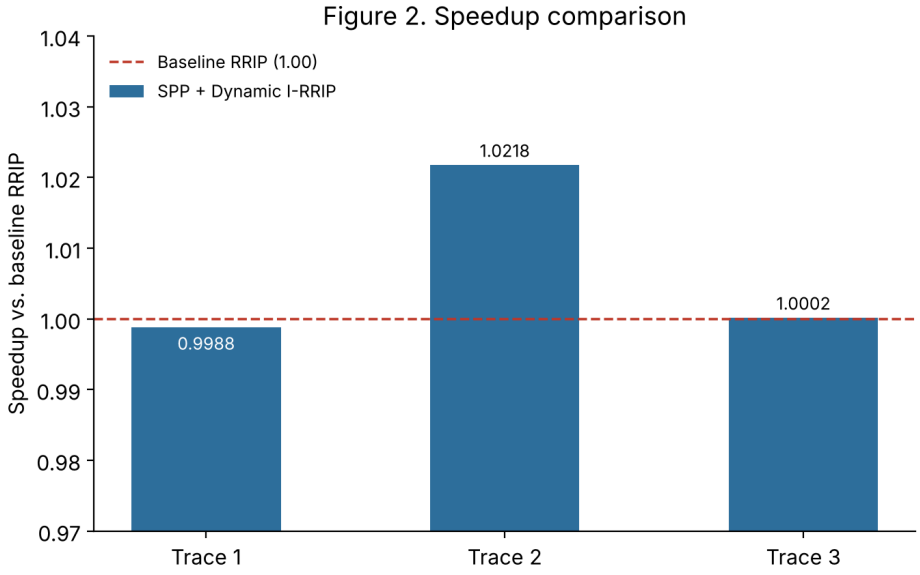


Figure 3. L2 cache load MPKI comparison

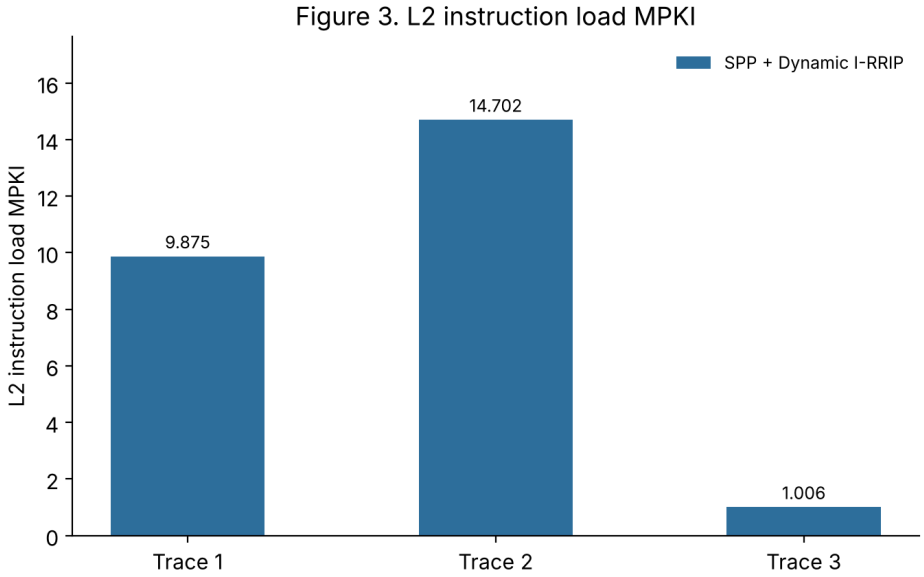
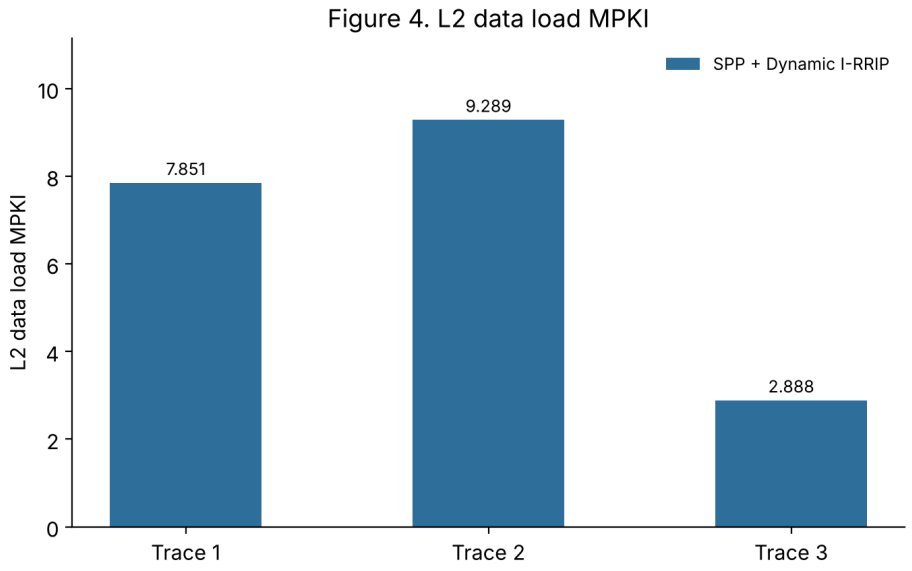


Figure 4. L2 instruction load MPKI comparison



8. Analysis and Discussion

8.1 Effect on IPC

The IPC results show that the combined SPP + Dynamic I-RRIP configuration achieves different performance levels across the three benchmarks. Trace 1 records an IPC of 0.231894, Trace 2 records 0.250030, and Trace 3 achieves the highest IPC of 0.381962.

Trace 3 has approximately 64.7% higher IPC than Trace 1 and 52.8% higher IPC than Trace 2. These differences reflect the measured performance of the configuration on the individual traces; they do not, by themselves, establish that the replacement policy provides a greater improvement over the baseline on Trace 3.

The speedup results provide a more direct comparison with baseline RRIP. Trace 1 achieves a speedup of 0.9988, which is approximately 0.12% below the baseline. Trace 2 achieves a speedup of 1.0218, representing an improvement of approximately 2.18%. Trace 3 achieves a speedup of 1.0002, corresponding to a very small improvement of approximately 0.02%.

These results indicate that the combined policy provides a measurable performance improvement over baseline RRIP for Trace 2, while its performance on Traces 1 and 3 is effectively similar to the baseline. In particular, the speedup of Trace 2 suggests that the combination of prefetching and dynamic replacement is more beneficial for this workload than for the other two traces.

However, the results provided cover only the combined configuration. To establish whether Task 3 meets the assignment's objective of outperforming both standalone SPP and Dynamic I-RRIP in IPC, their corresponding IPC values must also be compared against the combined configuration for each trace.

8.2 Effect on Prefetch Accuracy

Prefetch accuracy is important because it indicates how effectively the prefetcher predicts future memory accesses. A high accuracy generally means that a larger proportion of issued prefetches are useful, whereas low accuracy can indicate unnecessary prefetch activity and potential cache pollution.

In the combined design, prefetched blocks are inserted with the maximum RRPV of 3. This gives them a low initial retention priority, allowing them to become candidates for eviction before blocks with smaller RRPVs. The intention is to reduce the impact of prefetched blocks that are never subsequently accessed.

The benefit of this approach depends on the usefulness of the prefetched data. If a prefetched block is not needed, its low retention priority can help limit cache pollution. Conversely, if the block is accessed shortly after being prefetched, its low initial priority may cause it to be evicted before use.

The IPC results suggest that the impact of the combined design varies by workload, with the clearest improvement over baseline RRIP occurring in Trace 2. However, IPC alone cannot establish whether this difference is caused by prefetch accuracy or by changes in cache replacement behavior.

The definite result can only be obtained by using the actual results of the prefetch accuracy for SPP alone and for the proposed combination. However, since the actual prefetch accuracy was not provided in the tables given, no numerical result can be determined yet..

8.3 Effect on Coverage

Prefetch coverage measures the proportion of baseline L1D load misses eliminated by prefetching. It complements prefetch accuracy because a prefetcher may be accurate but cover only a small fraction of the total misses, or cover many misses while also issuing unnecessary prefetches.

The combined SPP + Dynamic I-RRIP policy aims to retain the benefits of SPP while reducing the cache pollution caused by unused prefetched blocks. By assigning prefetched lines a maximum RRPV at insertion, the policy attempts to prevent speculative data from displacing more valuable cache contents for an unnecessarily long period.

Nevertheless, the provided IPC and L2 MPKI values are insufficient to quantify the effect of this change on L1D prefetch coverage. Coverage must be calculated from the baseline and prefetch-enabled L1D load-miss counts, or taken from the corresponding simulator statistics.

The coverage results should therefore be interpreted alongside IPC and accuracy. Higher coverage would indicate that more baseline misses are avoided, but an increase in coverage would not necessarily translate into higher IPC if the prefetches arrive too late, create additional memory traffic, or cause useful cache lines to be evicted.

8.4 Effect on L2 Instruction and Data Misses

The L2 instruction and data Load MPKI results provide additional information about cache behavior under the combined policy.

Table 11: L2 instruction load MPKI

The instruction MPKI values are 9.87464 for Trace 1, 14.7016 for Trace 2, and 1.00612 for Trace 3. Trace 2 has the highest instruction MPKI, while Trace 3 has the lowest.

Table 12: L2 data load MPKI

The data MPKI values are 7.85076 for Trace 1, 9.28892 for Trace 2, and 2.88804 for Trace 3. As with instruction MPKI, Trace 2 records the highest value, whereas Trace 3 records the lowest.

Comparing the two categories reveals different miss patterns across the traces. For Trace 1, instruction MPKI is higher than data MPKI by approximately 2.02. For Trace 2, the difference is approximately 5.41, indicating a substantially higher instruction MPKI. For Trace 3, data MPKI is higher than instruction MPKI by approximately 1.88.

These results suggest that instruction misses are a particularly important component of L2 load-miss behavior in Traces 1 and 2. In Trace 3, data misses contribute more heavily than instruction misses. This distinction is relevant to the design because Dynamic I-RRIP attempts to protect instruction lines when instruction prioritization is enabled, while the prefetch-aware insertion rule gives newly prefetched blocks a low initial retention priority.

Trace 2 is particularly interesting: it has the highest instruction and data MPKI values but also achieves the largest speedup over baseline RRIP. This shows that a higher MPKI does not necessarily prevent a configuration from achieving a small performance improvement relative to a baseline. The absolute miss rate and the performance difference between configurations are separate measures.

However, because instruction and data MPKI values for baseline RRIP, standalone SPP, and Dynamic I-RRIP were not provided, these results cannot establish whether the combined design reduces instruction or data misses relative to the other configurations. The observed values describe the combined configuration only.

8.5 Overall Assessment

Considering the available results, the combined SPP + Dynamic I-RRIP configuration achieves its clearest improvement over baseline RRIP on Trace 2, with a speedup of 1.0218. Trace 1 is marginally below baseline performance, while Trace 3 is effectively at baseline performance.

The MPKI results also indicate that the three traces have different cache-miss characteristics. Traces 1 and 2 exhibit higher instruction MPKI than data MPKI, whereas Trace 3 exhibits higher data MPKI than instruction MPKI. These differences reinforce the importance of evaluating the replacement policy across multiple workloads rather than assuming that a single policy will provide the same benefit in every case.

The results are consistent with the objective of exploring a combination of prefetching and instruction-conscious replacement, but they do not establish that the integrated policy outperforms both standalone SPP and Dynamic I-RRIP. That conclusion requires direct comparison with the corresponding IPC, accuracy, coverage, and MPKI results for those configurations.

Overall, the measured speedups indicate that the combined policy has a small positive effect on Trace 2 but offers little or no improvement over baseline RRIP on the other two traces. Further comparison with the individual configurations is necessary to determine whether the prefetch-aware replacement modification provides a consistent advantage.